Εισαγωγή στην Python – Αναδρομή (recursion)

Αναδρομή

Μια συνάρτηση επιτρέπεται να καλεί μια άλλη συνάρτηση. Επίσης, μια συνάρτηση επιτρέπεται να καλεί τον εαυτό της. Μπορεί να μην είναι προφανές γιατί αυτό είναι καλό, αλλά αποδεικνύεται ότι είναι ένα από τα πιο μαγικά πράγματα που μπορεί να κάνει ένα πρόγραμμα.

```
def countdown(n):
    if n <= 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

Μια συνάρτηση που καλεί τον εαυτό της είναι **αναδρομική** (recursive). Η διαδικασία της εκτέλεσης της λέγεται **αναδρομή** (recursion).

```
def print_n(s, n):
    if n <= 0:
        return
    else:
        print(s)
        print_n(s, n-1)
```

Η περίπτωση όπου η συνθήκη n <= 0 ικανοποιείται ονομάζεται **βασική περίπτωση** (base case). Αν μια αναδρομή δεν φτάσει ποτέ σε μια βασική περίπτωση, συνεχίζει να κάνει αναδρομικές κλήσεις για πάντα, και το πρόγραμμα δεν τερματίζεται ποτέ. Αυτό είναι γνωστό ως ατέρμονη αναδρομή και γενικά το αποφεύγουμε:

```
def recurse():
    recurse()
```

Ασκήσεις

1. Ποια είναι η έξοδος του παρακάτω προγράμματος; Σχεδιάστε ένα διάγραμμα στοίβας που δείχνει την κατάσταση του προγράμματος όταν αυτό τυπώνει το αποτέλεσμα:

```
def recurse(n, s):
    if n == 0:
        print(s)
    else:
        recurse(n-1, n+s)

recurse(3, 0)
```

α. Τι θα συνέβαινε αν καλούσατε τη συνάρτηση ως εξής: `recurse(-1, 0)`;

β. Γράψε μια συμβολοσειρά τεκμηρίωσης (docstring) που θα εξηγεί όλα όσα πρέπει να γνωρίζει κάποιος για να χρησιμοποιήσει αυτή τη συνάρτηση (και τίποτα άλλο).

2. Διαβάστε την παρακάτω συνάρτηση και δείτε αν μπορείτε να καταλάβετε τι σχεδιάζει. Στη συνέχεια εκτελέστε την, π.χ. `draw(bob, 10, 5)` και δείτε αν την καταλάβατε σωστά.

```
def draw(t, length, n):
    if n == 0:
        return
    angle = 50
    t.fd(length*n)
    t.lt(angle)
    draw(t, length, n-1)
    t.rt(2*angle)
    draw(t, length, n-1)
    t.lt(angle)
    t.bk(length*n)
```

3. Σύμφωνα με το Τελευταίο θεώρημα του Fermat, δεν υπάρχουν θετικοί ακέραιοι a, b και c τέτοιοι ώστε

$$a^n + b^n = c^n$$

για οποιεσδήποτε τιμές του n μεγαλύτερες του 2.

α. Γράψτε μια συνάρτηση με όνομα `check_fermat` η οποία θα δέχεται τέσσερις παραμέτρους (a, b, c, n) και θα ελέγχει αν το θεώρημα του Fermat ισχύει. Αν το n είναι μεγαλύτερο από 2 και

$$a^n + b^n = c^n$$

το πρόγραμμα πρέπει να τυπώσει τη φράση Holy smokes, Fermat was wring! Σε αντίθετη περίπτωση, το πρόγραμμα πρέπει να τυπώσει τη φράση No, that doesn't work.

β. Γράψτε μια συνάρτηση που προτρέπει τον χρήστη να εισάγει τιμές για τα a, b, c και n, τις μετατρέπει σε ακέραιους αριθμούς, και χρησιμοποιεί την `check_fermat` για να ελέγξει αν αυτοί οι αριθμοί παραβιάζουν το θεώρημα του Fermat.